

Configurazione e modalità di consegna

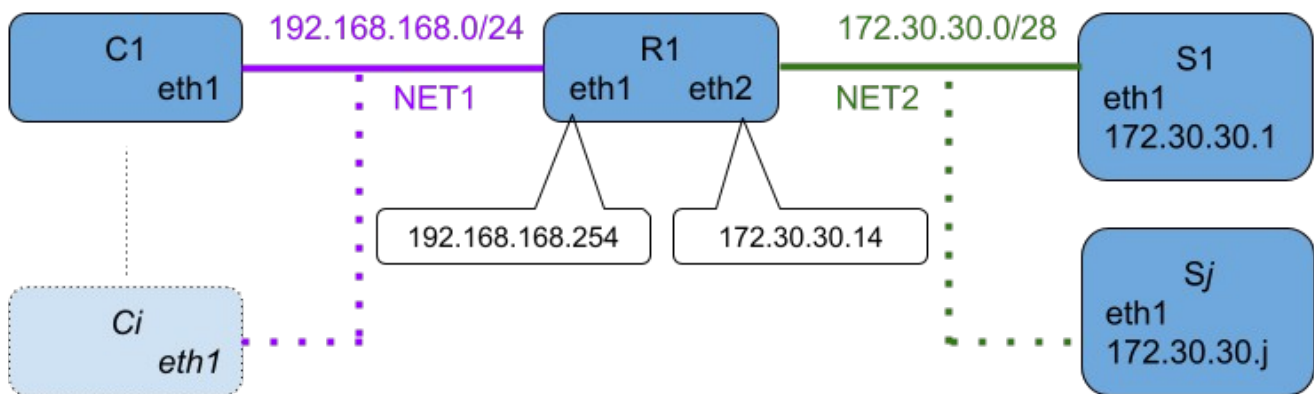
Create sulla macchina Linux host una cartella **ESAME_20260609** e all'interno di quella il **Vagrantfile** e una gerarchia standard di cartelle e file **Ansible** per generare le VM, configurarle, e trasferirvi i file necessari.

Per consegnare, create un archivio con nome **MATRICOLA.tar.gz**, in cui MATRICOLA sia il vostro numero su 10 cifre, e il tipo di file sia coerente con l'estensione, e inseritevi la cartella **ESAME_20260609** (e ovviamente tutto ciò che contiene).

Solo questo archivio deve essere consegnato via EOL. Nome, formato, struttura e contenuto dell'archivio saranno valutati.

Contesto dell'esercizio

Si consideri l'infrastruttura in figura con due reti **NET1** a **NET2** a cui sono rispettivamente connessi client **Ci** e server **Sj**



Lo scenario di utilizzo prevede che gli utenti possano utilizzare un Client per connettersi a un server ove svolgere una varietà di elaborazioni.

Per semplicità, si svolgano i test attivando **un client e due server**, ma gli script devono essere progettati in modo da funzionare anche per un numero arbitrario di client e di utenti.

Di seguito sono dettagliate le attività da svolgere.

Leggere tutto il testo per dedurre dagli script quali configurazioni siano indispensabili per il loro funzionamento, e predisporle in modo appropriato.

Rete e servizi da predisporre

R1 deve avere le seguenti caratteristiche

- indirizzo statico **192.168.168.254/24** su **NET1**
- indirizzo statico **172.30.30.14/28** su **NET2**
- eroga via DHCP ai Client indirizzi nel range equivalente alla sottorete **192.168.168.0/25**
- eroga via DHCP ai Server tutti gli indirizzi disponibili nella sottorete **172.30.30.0/28**

Deve essere garantito l'instradamento tra i Client e i Server.

I soli sistemi Client e Server devono essere configurati per il login centralizzato via LDAP. Il directory server è in esecuzione su **R1**.

Su **R1** i messaggi ricevuti via syslog con etichetta **local3.notice** vengono scritti nel file **/var/log/servermem.log**

Con Ansible, si creino in LDAP

- un gruppo **workers** con id=5000
- un utente **worker-aa** con id=5001, password **notsecure**, shell **/bin/bash** e con gruppo principale **workers**

Ancora con Ansible, si crei la home directory **/home/worker-aa** sui Client e i Server, e si predispongano tali host in modo che l'utente possa connettersi via SSH dai Client ai Server senza bisogno di password, usando una chiave preparata staticamente sull'host di controllo

Script

Si realizzi uno script **/usr/sbin/stats.sh** collocato sui Server

Lo script, eseguibile solo da **root**, viene avviato automaticamente al boot e deve essere riavviato in caso di terminazione anomala.

Mantiene una statistica di utilizzo della memoria secondo un modello a "finestra scorrevole" in questo modo:
ogni 15 secondi

- ricava la quantità di memoria RAM libera in kB
- la memorizza in coda al file **/var/run/freemem**
- se nel file sono presenti più di 50 righe, tiene solo le ultime 50 ed elimina le più vecchie

Si realizzi uno script **/usr/sbin/average.sh** collocato sui Server

Lo script, lanciato automaticamente da **root** ogni 2 minuti in orario lavorativo (8:00-19:58 da lunedì a venerdì),

- legge il contenuto di **/var/run/freemem**
- calcola la media dei valori contenuti- se il file è vuoto o inesistente, imposta *media*=0
- invia **_IPSERVER_media_** via syslog a **R1** con etichetta **local3.notice**

Si realizzi uno script **/usr/sbin/inform.sh** collocato su **R1**

Lo script **inform.sh** viene eseguito dall'agent **SNMP**

- ricava da **/var/log/servermem.log** le righe con timestamp non più vecchio di 2 minuti
- se non esistono righe del log con timestamp abbastanza recenti, restituisce la stringa **ERROR**
- altrimenti individua tra queste il server con più memoria libera e ne restituisce l'indirizzo

Si realizzi uno script **/usr/bin/connect.sh** collocato sui **Client**

Lo script deve essere eseguibile solo dai membri del gruppo **workers**

- Verifica che sia stato passato almeno un parametro sulla riga di comando
- Chiede interattivamente all'utente di inserire una stringa **OUTPUT** composta di soli caratteri minuscoli; se l'utente inserisce una stringa vuota, termina, altrimenti ripete la richiesta.
- Interroga **R1** via **SNMP** per ricavare l'indirizzo **IPSERVER** del server con più memoria libera
 - se la risposta è **ERROR**, riporta a schermo l'errore e termina,
 - altrimenti:
- aggiorna la entry **LDAP** dell'utente che lo sta eseguendo, in modo che contenga un valore dell'attributo di tipo **description** nel formato **IPSERVER:NUMEROCONNESSIONI**; in dettaglio:
 - se non esiste un valore di tipo **description** che menziona **IPSERVER**, aggiungerà tra i valori **IPSERVER:1**
 - se esiste, sostituirà il valore presente incrementando di 1 il numero di connessioni
- esegue via **SSH** sul server indicato il comando (coi relativi parametri) specificato sulla riga di comando, salvando l'output nel file **~/OUTPUT** sul Server. Lo script non deve attendere la terminazione del comando, ma deve lanciare immediatamente **/usr/bin/recover.sh** passandogli **IPSERVER**, **OUTPUT** e il PID del processo **ssh** come parametri

Si realizzi uno script **/usr/bin/recover.sh** collocato sui **Client**

- Riceve come parametri IP del Server, il nome del file di **OUTPUT**, e il PID del processo **ssh**
- Tenta di copiare il file dal Server al Client locale
- Se trova il file vuoto o in caso di errore, si ri-schedula per essere eseguito dopo 10 minuti,
- Deve ri-schedularsi per un massimo di 5 tentativi; se li raggiunge senza successo, scrive **"FAILED"** nel file di output e termina (se ancora attivo) il processo **ssh**